

Machine Learning

Boosting Techniques: XGBoost & LightGBM



Dr. G. Omprakash

KLEF Hyderabad (Aziz Nagar) – Dept. of ECE

Machine Learning (25SC2107E)

Session Outcome

- Understand how weak models added one at a time — *AdaBoost*, *gradient boosting*, *XGBoost*, *LightGBM* — become a very strong model.

XGBoost

Key Terms — XGBoost and LightGBM

Definitions

Gradient g_i , Hessian h_i — the first and second derivatives of the loss with respect to the current prediction $\hat{y}_i$. XGBoost uses both; plain gradient boosting uses only g_i .

Leaf weight w^* — the value a leaf outputs, $w^* = -G/(H + \lambda)$, where G and H are the sums of g_i and h_i over the rows in that leaf.

Regularisation λ, α, γ — λ (L2) shrinks every leaf; α (L1) can set a leaf to exactly zero; γ is the minimum gain a split must earn to be made.

Histogram binning — bucketing each feature into ≤ 256 bins once, then testing only the bin edges instead of all $n - 1$ thresholds.

Level-wise vs leaf-wise growth — splitting every node at a depth before going deeper (XGBoost), versus always splitting the single highest-gain leaf anywhere in the tree (LightGBM).

Default direction — the side a node sends rows whose value is missing, learned from the data rather than imputed.

XGBoost stands for **Extreme Gradient Boosting**.

We build trees one after another. Each new tree tries to fix the errors left behind by the trees already built. The final prediction is the sum of what all the trees say:

$$\hat{y}_i = \sum_{k=1}^K f_k(x_i)$$

- K is the number of trees
- f_k is the k -th tree, which takes an input x_i and returns a number

Question: how do we decide what each new tree should look like?

What XGBoost tries to make small

XGBoost tries to minimise the objective function:

$$\text{Objective} = \underbrace{\sum_{i=1}^n \text{error}_i}_{\text{how wrong we are}} + \underbrace{\sum_{k=1}^K \text{penalty}(f_k)}_{\text{how complicated the trees are}}$$

For a regression problem the error for one data point is simply

$$\text{error}_i = \frac{1}{2} (y_i - \hat{y}_i)^2$$

that is, the squared gap between the actual value and the predicted value.

The second part is what makes XGBoost different from plain gradient boosting.

For a single tree with T leaves and leaf values $w_1, \dots, w_T$:

$$\text{penalty}(f) = \gamma T + \frac{1}{2} \lambda \sum_{j=1}^T w_j^2$$

- γ is the price paid for *every extra leaf*
- λ is the price paid for *large leaf values*

So a tree is not allowed to grow freely. It must earn every leaf it adds. This is what stops XGBoost from building huge trees that simply memorise the training data.

Let $\hat{y}_i^{(t-1)}$ be the prediction after $t - 1$ trees. Adding the new tree f_t gives

$$\hat{y}_i^{(t)} = \hat{y}_i^{(t-1)} + f_t(x_i)$$

So at step t we want to minimise

$$\text{Obj}^{(t)} = \sum_{i=1}^n \frac{1}{2} \left(y_i - [\hat{y}_i^{(t-1)} + f_t(x_i)] \right)^2 + \text{penalty}(f_t)$$

The only unknown is the new tree f_t . (Everything from the earlier trees is already fixed)

Using Taylor Series for the objective

Let the gradient be $g_i = \hat{y}_i^{(t-1)} - y_i$ and hessian $h_i = 1$ (for squared error)

Using g_i and h_i the objective becomes approximately

$$\text{Obj}^{(t)} \approx \sum_{i=1}^n \left[g_i f_t(x_i) + \frac{1}{2} h_i f_t(x_i)^2 \right] + \gamma T + \frac{1}{2} \lambda \sum_{j=1}^T w_j^2$$

This is now a quadratic function which means we can solve it exactly.

Grouping by leaves:

$$G_j = \sum_i g_i, \quad H_j = \sum_i h_i$$

The objective function is given by

$$\text{Obj}^{(t)} \approx \sum_{j=1}^T \left[G_j w_j + \frac{1}{2} (H_j + \lambda) w_j^2 \right] + \gamma T$$

Optimal value for a leaf

Differentiate the leaf term with respect to w_j , set it to zero:

$$w_j^* = - \frac{G_j}{H_j + \lambda}$$

Why *Extreme*?

- Feature values are sorted once and stored in a compressed column format, so candidate splits are evaluated in parallel across features.
- Missing values are handled automatically: for each split the algorithm learns which side missing entries should go to.
- Rows and columns can be subsampled, borrowing an idea from random forests, giving further protection against overfitting.

How a split is chosen

Suppose we consider splitting one leaf into a left part L and a right part R . The improvement is

$$\text{Gain} = \frac{1}{2} \left[\frac{G_L^2}{H_L + \lambda} + \frac{G_R^2}{H_R + \lambda} - \frac{(G_L + G_R)^2}{H_L + H_R + \lambda} \right] - \gamma$$

- first two terms: scores of the two new leaves
- third term: score of the leaf before splitting
- γ : the cost of adding one more leaf

XGBoost tries every feature and every cut point, and takes the best gain. If the best gain is negative, it does not split at all.

Quick Check — Gradients, Hessians, Leaf Weights

Direct questions

- ❶ For $L(y, \hat{y}) = \frac{1}{2}(y - \hat{y})^2$, give g_i and h_i .

Ans. $g_i = \frac{\partial L}{\partial \hat{y}_i} = -(y_i - \hat{y}_i) = -r_i$ (minus the residual), and $h_i = 1$.

- ❷ With $h_i = 1$ and $\lambda = 0$, what does $w^* = -G/(H + \lambda)$ reduce to?

Ans. $w^* = -\frac{\sum g_i}{n} = \frac{\sum r_i}{n}$ = the mean residual in the leaf (ordinary gradient boosting is the special case $\lambda = 0$).

- ❸ A leaf has $G = -6$, $H = 3$. Give w^* for $\lambda = 0$ and for $\lambda = 3$.

Ans. $\lambda = 0$: $w^* = 6/3 = 2.0$. $\lambda = 3$: $w^* = 6/6 = 1.0$ — the penalty halved the leaf.

Driver Value at a Ride-Hailing Firm

Driver Value

A ride-hailing company predicts each driver's next-quarter earnings contribution with XGBoost and a custom squared-error objective. The data sits across several machines and is too large to sort in memory.

- (a) Explain why a single deep regression tree is unstable when a few training rows change, and why boosting deliberately uses shallow trees instead.
- (b) For the custom loss $L(y, \hat{y}) = \frac{1}{2} (y - \hat{y})^2$, derive the first-order gradient g_i and the second-order Hessian h_i , and show how XGBoost turns them into the optimal leaf weight. Then compute that weight for a leaf holding three drivers with residuals $+4, -1, +3$ and $\lambda = 2$.

Answer (a) and (b)

(a) Instability, and why boosting wants weak trees

Why a deep tree is unstable

A tree picks each split greedily, so when two candidate splits score almost equally, changing a few rows can flip the winner. Since every split below is fitted inside the region its parent created, one flip at the top redraws the whole subtree beneath it. Small data change, large tree change, which is high variance. Deep trees also leave very few drivers per leaf, so each leaf value rests on almost no data.

Why boosting uses shallow trees

Bagging wants deep trees, because averaging many high variance trees cancels the variance out. Boosting reduces bias instead, by adding small corrections one at a time, so each tree should be high bias and low variance, usually depth 2 to 4. A deep first tree would leave almost no error for the later trees to correct, and its noise would be permanently added into the sum, since boosting only adds and never revises.

(b) Derivation and the numeric leaf weight

$$L = \frac{1}{2} (y_i - \hat{y}_i)^2 \Rightarrow g_i = \frac{\partial L}{\partial \hat{y}_i} = -(y_i - \hat{y}_i) = -r_i, \quad h_i = \frac{\partial^2 L}{\partial \hat{y}_i^2} = 1. \text{ Minimising } Gw + \frac{1}{2} (H + \lambda) w^2 \text{ over } w \text{ gives } \boxed{w^* = -\frac{G}{H + \lambda}} \text{ with}$$

$G = \sum g_i, H = \sum h_i$. Residuals +4, -1, +3 $\Rightarrow g = (-4, +1, -3)$, so $G = -6$; $h_i = 1$ each, so $H = 3$.

$$w^* = -\frac{-6}{3 + 2} = \frac{6}{5} = +1.2$$

Without the penalty it would be $6/3 = 2.0$: $\lambda = 2$ pulls the leaf from 2.0 down to 1.2.

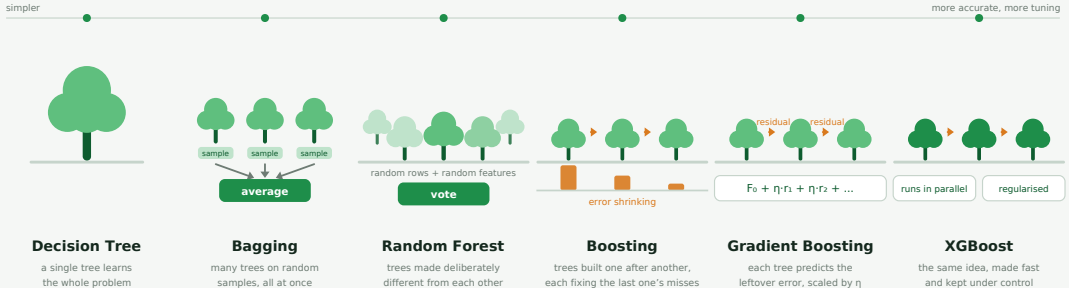
XGBoost in Code

```
1 import xgboost as xgb
2
3 model = xgb.XGBClassifier(
4     n_estimators=200,
5     learning_rate=0.05,
6     max_depth=3,
7     reg_lambda=1.0,    # L2 on leaves
8     gamma=0.0,         # min gain to split
9     subsample=0.8,
10    colsample_bytree=0.8,
11    eval_metric="logloss",
12    early_stopping_rounds=20)
13
14 model.fit(Xtr, ytr,
15         eval_set=[(Xte, yte)],
16         verbose=False)
```

- `reg_lambda` — shrinks the leaf values.
- `gamma` — the minimum gain a split must earn.
- `colsample_bytree` — the forest trick, on columns.
- `early_stopping_rounds` — stop when the eval score stalls.

Evolution of Tree Algorithms

one tree, then many in parallel, then many in sequence



LightGBM

LightGBM

LightGBM stands for *Light Gradient Boosting Machine*. It solves the same problem as XGBoost, but it is built for speed on large datasets.

The prediction is still a sum of trees, added one at a time:

$$\hat{y}_i = \sum_{k=1}^K f_k(x_i), \quad \hat{y}_i^{(t)} = \hat{y}_i^{(t-1)} + \eta f_t(x_i)$$

The objective is also the same:

$$\text{Objective} = \underbrace{\sum_{i=1}^n \text{error}_i}_{\text{how wrong we are}} + \underbrace{\sum_{k=1}^K \text{penalty}(f_k)}_{\text{how complicated the trees are}}$$

So what is different? **How each tree is built.**

The same two numbers per data point

As before, each data point is summarised by two numbers. For squared error:

$$g_i = \hat{y}_i^{(t-1)} - y_i, \quad h_i = 1$$

Grouping by leaves with $G_j = \sum_{i \in I_j} g_i$ and $H_j = \sum_{i \in I_j} h_i$, the best leaf value and the split gain are unchanged:

$$w_j^* = -\frac{G_j}{H_j + \lambda}, \quad \text{Gain} = \frac{1}{2} \left[\frac{G_L^2}{H_L + \lambda} + \frac{G_R^2}{H_R + \lambda} - \frac{(G_L + G_R)^2}{H_L + H_R + \lambda} \right] - \gamma$$

LightGBM keeps this mathematics. Its contribution is making the search for good splits far cheaper.

The bottleneck: finding the split

To choose one split, the exact method must:

- sort every feature's values,
- walk through every possible cut point,
- recompute G_L , H_L , G_R , H_R at each one.

With n rows and d features this costs roughly $O(np)$ per split, and the sorted values must sit in memory.

For millions of rows this is the expensive step. LightGBM uses two methods:

- Histogram Binning
- Leaf-wise growth

Histogram binning

Instead of testing every distinct value of a feature, LightGBM first groups the values into a small number of *bins*, typically 255.

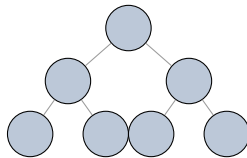
For each bin b it stores only two running totals:

$$G_b = \sum_{i \in b} g_i, \quad H_b = \sum_{i \in b} h_i$$

A split is then found by scanning bins and accumulating these totals, so the cost per split drops from $O(np)$ to $O(\#bins \times p)$.

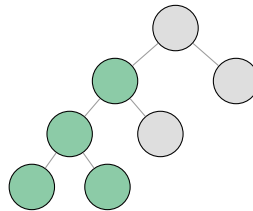
Leaf-wise growth

Level-wise (XGBoost)



fill each level
before going deeper

Leaf-wise (LightGBM)



always split the leaf that
reduces the loss most

Leaf-wise growth

Level-wise (XGBoost default)

- grows every node at a depth before moving deeper
- balanced tree
- some splits add very little

Leaf-wise (LightGBM)

- always splits the single leaf with the largest gain
- unbalanced, deeper in useful places
- fewer leaves for the same accuracy

The trade-off

Leaf-wise growth reduces the loss faster, but it can grow very deep and overfit on small datasets. LightGBM therefore caps it with `num_leaves` and `max_depth`.

Quick Check — Binning and Regularisation

Direct questions

- ❶ A feature has 100,000 distinct values. Split points to evaluate, exact vs 256 bins?

Ans. Exact: 99,999. Histogram: 255. About 392× fewer.

- ❷ Why does binning also cut memory?

Ans. A bin index fits in 8 bits; the original value needed 32 or 64. Roughly a 4–8× saving, which is what lets the whole dataset stay in cache.

Nightly Retraining at a Logistics Network

Case study

A logistics network scores 4 million parcel scans a day for mis-routing and must retrain every night. LightGBM and XGBoost with histogram-based splitting are chosen to meet the latency budget. Mis-routing is rare: only 0.4% of scans.

- (a) Explain the mechanism of histogram-based feature binning, and how it changes split-finding complexity from $O(N \cdot p)$ to $O(n_{\text{bins}} \cdot p)$.
- (b) A numerical feature `TransitHours` contains $N = 2,000,000$ unique floating-point values. Calculate the maximum number of split evaluation points required by exact greedy splitting versus a 256-bin histogram approach, for that one feature.

Case study: Answer (a) and (b)

(a) The binning mechanism

How binning works

Each feature's values are grouped once into a small fixed number of bins, usually 255. For each bin, only two totals are kept, the sum of gradients G_b and the sum of curvatures H_b . After that the raw values are not needed, only each row's bin index.

Why the complexity drops

The exact method sorts all N rows per feature and tests every cut point, recomputing the left and right sums each time, costing $O(N \cdot p)$ per split. With histograms, the scan just walks across bins accumulating the stored totals, and the right side comes free by subtracting from the parent: $G_R = G_{\text{parent}} - G_L$, $H_R = H_{\text{parent}} - H_L$. Only n_{bins} positions per feature need checking, so the cost becomes $O(n_{\text{bins}} \cdot p)$.

For 4 million scans with 255 bins, that is roughly a 15000 fold cut in work per split, which is what makes the nightly retrain fit the budget. A further saving comes from building the histogram of only the smaller child and getting the larger one by subtraction.

(b) The arithmetic for TransiHours

Split evaluation points		
Exact greedy	$N - 1 = 2,000,000 - 1 = 1,999,999$	(one per gap between sorted values)
256-bin histogram	$256 - 1 = 255$	(one per bin edge)
Reduction	$1,999,999 / 255 \approx 7,843 \times \text{fewer}$	

Why Trees Beat Linear Models and Nets on Messy Tables

1. Invariance

A split only asks whether a value is above or below a threshold, so rescaling, skew, log transforms and outliers change nothing. One income mis-typed as 10^9 is just a row on one side of the cut. In a linear model that single row drags the whole fitted coefficient.

2. Free non-linearity

Every root-to-leaf path is a chain of conditions joined by **and**, so ranges and interactions come for free. A linear model needs them written in by hand, and a neural net has to build a sharp step out of many smooth pieces.

3. Right inductive bias

Mixed column types and missing values are handled directly. A table has no spatial or sequential structure for a network to exploit, so a net must learn from scratch what a tree already assumes, and it needs far more rows to get there.

This is why gradient-boosted trees are still the default first model for **tabular** data.

Scenario — Insurance Claim Cost on a Legacy Table

Scenario S6

An insurer models **claim cost** from a legacy table: mixed data types, 18% of **VehicleAge** **missing**, a heavy right tail of very large claims, and strongly non-linear no-claim-bonus **bands**. The team must choose between a logistic/linear baseline, a neural network, and XGBoost / LightGBM.

- (a) Give **three structural reasons** why gradient-boosted trees routinely outperform linear models and deep neural networks on uncleaned tabular data.
- (b) Demonstrate how XGBoost handles the 18% missing **VehicleAge** values during split candidate evaluation, **without** imputing them beforehand.
- (c) Compare **level-wise** growth in XGBoost with **leaf-wise** growth in LightGBM, stating how leaf-wise reduces loss faster and what constraint is needed to stop it overfitting.

Scenario S6 — Answer (a) and (b)

(a) Three structural reasons

1. Invariance to scale and outliers. Splits use only the order of the values, so the heavy right tail of claim amounts needs no log-transform or winsorising, and no feature needs standardising. A linear model's coefficient is pulled by every extreme value; a tree confines it to one leaf. **2. Thresholds and interactions are native.** The no-claim-bonus bands are thresholds, and each root-to-leaf path is a conjunction of them — exactly the hypothesis class a tree represents. A linear model would need every band and interaction hand-coded; an MLP must build a step out of many smooth units. **3. Mixed types, missing values, and the right prior.** Categories and gaps are handled inside the algorithm. Beyond that, a table has no locality or translation structure for a network's architecture to exploit, so the network has no head start and needs far more data to reach the same place.

(b) Missing VehicleAge: the learned default direction

Rows with a missing value are simply **left out of the sorted scan**. For each candidate threshold the gain is then computed **twice**: once sending all missing rows to the **left** child, once sending them all to the **right**. Whichever yields the higher gain is kept, and that side is stored on the node as its **default direction**; at prediction time any row with a missing value follows it. Cost: two passes instead of one. Benefit: no imputation, and the fact of being missing becomes usable signal — if unregistered older vehicles are the ones with no recorded age, the model can route them profitably, which mean-imputation would have erased.

Scenario S6 — Answer (c)

Level-wise (XGBoost default)

split every node at depth d before depth $d + 1$

balanced, symmetric trees

controlled by `max_depth`

some splits earn little — wasted work

stable, hard to overfit

Leaf-wise (LightGBM)

always split the highest-gain leaf anywhere

deep, lop-sided trees

controlled by `num_leaves`

every split is the best one available

fast, but overfits small data

(c) Faster loss reduction — and the constraint it needs

For a **fixed budget of leaves**, leaf-wise growth reduces the loss more, because it spends every leaf on the single best split available anywhere in the tree, while level-wise is obliged to also make the poor splits that complete the level. The cost is that the tree grows deep along one branch and can isolate a handful of rows. **The necessary constraint** is a cap on `num_leaves` (keep `num_leaves` $< 2^{\text{max_depth}}$), backed by `min_data_in_leaf` and, as a backstop, `max_depth`. On a small insurance table, unconstrained leaf-wise growth is the classic LightGBM overfitting trap — prefer XGBoost there.

The Four, Side by Side

	AdaBoost	Gradient B.	XGBoost	LightGBM
Focuses mistakes by	row weights	residuals	residuals	residuals
Uses curvature	no	no	yes	yes
Tree growth	level-wise	level-wise	level-wise	leaf-wise
Split search	exact	exact	exact or hist	histogram
Regularisation	none	shrinkage	built in	built in
Missing values	no	no	yes	yes
Best for	teaching	medium data	most problems	large data

- First attempt on tabular data: XGBoost or HistGradientBoosting. Very large data: LightGBM.

Bagging or Boosting?

Random Forest — bagging

trees built in parallel

deep trees

fixes variance

more trees always safe

very little tuning

test 0.838

Boosting

trees built in a queue

shallow trees

fixes bias

more trees can overfit

a fair amount of tuning

test 0.825–0.838

Summary — Boosting

